

03a Baseline Time Series Models

Objective: benchmark classical baseline models for monthly CFPB credit reporting share using the trimmed 2012-01-01 to 2025-12-31 window and identify the strongest interpretable baselines for the 2026 outlook.

1. Set Up Data

- Dataset: `../data/processed/credit_reporting_monthly_data.csv`
- Scope: products whose label starts with `Credit reporting` (merged)
- Date window: `2012-01-01` to `2025-12-31`
- Frequency: monthly credit reporting share (%)
- Dependent variable: `credit_reporting_share_pct`
- Split: fixed train/validation/test with validation and test sized as evenly as possible
- Forecast horizon: 12 months

```
In [1]: # =====
# IMPORT STATEMENTS
# =====
# Import core libraries for time series analysis, numerical computation, and visual

import time
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from itertools import product

# Import statsmodels forecasting and diagnostics modules
from statsmodels.tsa.holtwinters import ExponentialSmoothing, SimpleExpSmoothing
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.forecasting.theta import ThetaModel
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.stats.diagnostic import acorr_ljungbox

warnings.filterwarnings("ignore")
```

```
In [2]: # =====
# SECTION 1: DATA SETUP
# =====
# Load monthly credit reporting share series and create train/val/test splits.

FREQ_ALIAS = "M"
SEASONAL_PERIODS = 12
FORECAST_HORIZON = 12
```

```

df = pd.read_csv("../data/processed/credit_reporting_monthly_data.csv")
df["month"] = pd.PeriodIndex(df["month"], freq=FREQ_ALIAS)
df = df.sort_values("month")

series_df = (
    df[["month", "credit_reporting_share_pct"]]
    .rename(columns={"credit_reporting_share_pct": "y"})
    .set_index("month")
    .astype(float)
)
y = series_df["y"].copy()

n = len(y)
test_size = max(4, n // 5)
val_size = test_size
while n - (val_size + test_size) < 8 and test_size > 2:
    test_size -= 1
    val_size = test_size

train = y.iloc[: n - val_size - test_size].copy()
val = y.iloc[n - val_size - test_size : n - test_size].copy()
test = y.iloc[n - test_size :].copy()

print(f"Frequency: {FREQ_ALIAS} | Train={len(train)}, Val={len(val)}, Test={len(test)}")
display(series_df.head())

```

Frequency: M | Train=103, Val=34, Test=34

y	
month	
2011-12	0.0
2012-01	0.0
2012-02	0.0
2012-03	0.0
2012-04	0.0

2. Define Helpers

This section defines reusable utility functions and global state for the forecasting pipeline. We implement three categories of helpers:

1. **Metric Functions** (sMAPE, MAE, RMSE): Standard evaluation metrics for comparing forecast accuracy across all models
2. **Baseline Forecast Functions** (Naive, Moving Average, Seasonal Naive, FFT): Simple, interpretable methods that serve as strong benchmarks
3. **Diagnostics Functions** (spectral entropy, residual diagnostics, plotting): Tools to assess model quality and visualize predictions

These helpers are applied consistently across all model families in Sections 3–7, enabling fair comparison.

```
In [3]: # =====
# SECTION 2: HELPER FUNCTIONS & UTILITIES
# =====

# ---- Metric Functions ----
def smape(y_true, y_pred):
    """Compute Symmetric Mean Absolute Percentage Error (sMAPE)"""
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)
    denom = np.abs(y_true) + np.abs(y_pred)
    ratio = np.where(denom == 0, 0.0, 2.0 * np.abs(y_true - y_pred) / denom)
    return float(100.0 * np.mean(ratio))

def mae(y_true, y_pred):
    """Compute Mean Absolute Error (MAE)"""
    return float(np.mean(np.abs(np.asarray(y_true) - np.asarray(y_pred))))

def rmse(y_true, y_pred):
    """Compute Root Mean Squared Error (RMSE)"""
    return float(np.sqrt(np.mean((np.asarray(y_true) - np.asarray(y_pred)) ** 2)))

# ---- Baseline Forecast Functions ----
def fc_naive(train_series, h):
    """Naive forecast: repeat last observed value h times"""
    return np.repeat(train_series.iloc[-1], h)

def fc_mavg(train_series, h, k=4):
    """Moving average forecast: repeat the mean of last k values h times"""
    k_eff = min(k, len(train_series))
    return np.repeat(train_series.iloc[-k_eff:].mean(), h)

def fc_snaive(train_series, h, season=SEASONAL_PERIODS):
    """Seasonal naive forecast: repeat the last seasonal cycle"""
    if len(train_series) < season:
        return fc_naive(train_series, h)
    last_season = train_series.iloc[-season:].values
    reps = int(np.ceil(h / season))
    return np.tile(last_season, reps)[:h]

def fc_fft(train_series, h, keep=6):
    """FFT-based forecast: retain top k frequency components for reconstruction"""
    values = np.asarray(train_series, dtype=float)
    n = len(values)
    x = np.arange(n, dtype=float)

    # Step 1: Remove Linear trend before FFT, then add it back after reconstruction
    slope, intercept = np.polyfit(x, values, 1)
    trend = slope * x + intercept
    detr = values - trend

    # Step 2: Compute FFT and identify top k dominant frequencies
    fft_vals = np.fft.rfft(detr)
```

```

amps = np.abs(fft_vals)
keep = min(keep, len(amps))
idx = np.argsort(amps)[-keep:] # Indices of largest amplitudes

# Step 3: Filter FFT to keep only top k components
filt = np.zeros_like(fft_vals, dtype=complex)
filt[idx] = fft_vals[idx]

# Step 4: Reconstruct signal into future horizon and add back trend
x_future = np.arange(n + h, dtype=float)
recon = np.zeros(n + h, dtype=float)
freqs = np.fft.rfftfreq(n, d=1.0)
for k, f in enumerate(freqs):
    c = filt[k]
    if np.abs(c) == 0:
        continue
    recon += (2.0 / n) * np.real(c * np.exp(2j * np.pi * f * x_future))

trend_future = slope * x_future + intercept
out = recon + trend_future
return out[-h:] # Return only future h steps

# ---- Results & Registry Functions ----
def append_result(model_name, val_pred, test_pred, runtime_sec, fit_obj=None):
    """Store model evaluation metrics and predictions in global registry"""
    eval_rows.append({
        "Model": model_name,
        "sMAPE_val": smape(val.values, val_pred),
        "MAE_val": mae(val.values, val_pred),
        "RMSE_val": rmse(val.values, val_pred),
        "sMAPE_test": smape(test.values, test_pred),
        "MAE_test": mae(test.values, test_pred),
        "RMSE_test": rmse(test.values, test_pred),
        "runtime_sec": runtime_sec
    })
    model_registry[model_name] = {
        "fit": fit_obj,
        "val_pred": pd.Series(val_pred, index=val.index, name=model_name),
        "test_pred": pd.Series(test_pred, index=test.index, name=model_name)
    }

# ---- Visualization Functions ----
def plot_section_predictions(section_title, pred_map):
    """Plot observed train/val/test series alongside model predictions for a section
    fig, ax = plt.subplots(figsize=(10, 4))

    # Convert period index to timestamp for plotting
    y_train_ts = pd.Series(train.values, index=train.index.to_timestamp(how="end"))
    y_val_ts = pd.Series(val.values, index=val.index.to_timestamp(how="end"))
    y_test_ts = pd.Series(test.values, index=test.index.to_timestamp(how="end"))

    # Plot observed data with different line styles for each period (train/val/test
    ax.plot(y_train_ts.index, y_train_ts.values, color="steelblue", label="Observed
    ax.plot(y_val_ts.index, y_val_ts.values, color="steelblue", linestyle="--", al
    ax.plot(y_test_ts.index, y_test_ts.values, color="steelblue", linestyle=":", al

```

```

# Overlay model predictions for both val and test periods
for model_name, preds in pred_map.items():
    val_ts = pd.Series(preds["val"], index=val.index.to_timestamp(how="end"))
    test_ts = pd.Series(preds["test"], index=test.index.to_timestamp(how="end"))
    ax.plot(val_ts.index, val_ts.values, linestyle="--", marker="o", label=f"{model_name} - Val")
    ax.plot(test_ts.index, test_ts.values, marker="o", label=f"{model_name} - Test")

ax.set_title(section_title)
ax.legend(ncol=2, fontsize=8)
plt.tight_layout()
plt.show()

# ---- Diagnostics Functions ----
def compute_spectral_entropy(series):
    """Compute normalized spectral entropy (0-1 scale; higher = less predictable)"""
    x = np.asarray(series, dtype=float)
    x = x - np.mean(x) # Center the data
    psd = np.abs(np.fft.rfft(x)) ** 2 # Power Spectral Density
    if psd.sum() == 0:
        return 0.0
    p = psd / psd.sum() # Normalize to probability distribution
    p = p[p > 0] # Remove zero probabilities for entropy calculation
    h = -np.sum(p * np.log2(p)) # Shannon entropy
    h_max = np.log2(len(psd)) if len(psd) > 1 else 1.0 # Maximum possible entropy
    return float(h / h_max) # Normalized entropy

def run_residual_diagnostics(df_results, registry, top_n=2):
    """Extract residual diagnostics (mean, std, Ljung-Box test) for top N models"""
    top_models = df_results.head(top_n)["Model"].tolist()
    rows = []
    for m in top_models:
        fit_obj = registry[m]["fit"]
        if fit_obj is not None and hasattr(fit_obj, "resid"):
            resid = pd.Series(fit_obj.resid).dropna()
            # Ljung-Box test for residual autocorrelation at Lag 4 (seasonal)
            lb_p = float(acorr_ljungbox(resid, lags=[4], return_df=True)["lb_pvalue"])
            rows.append({
                "Model": m,
                "resid_mean": float(resid.mean()),
                "resid_std": float(resid.std(ddof=1)),
                "ljung_box_p_lag4": lb_p
            })
        else:
            rows.append({
                "Model": m,
                "resid_mean": np.nan,
                "resid_std": np.nan,
                "ljung_box_p_lag4": np.nan
            })
    return pd.DataFrame(rows)

# ---- Global State: Tracking Results & Models ----
eval_rows = [] # List of evaluation metric dicts (populated by each section)
model_registry = {} # Dict storing fit objects and predictions for all models

```

3. Evaluate Simple Baselines (Naive, MA, Seasonal Naive)

This section evaluates three deterministic monthly benchmarks that require no parameter fitting and therefore act as fast reference floors:

- **Naive:** repeats the last observed month
- **Moving Average (MA):** repeats the mean of the last 4 months
- **Seasonal Naive (SNaive):** repeats the last 12-month seasonal cycle

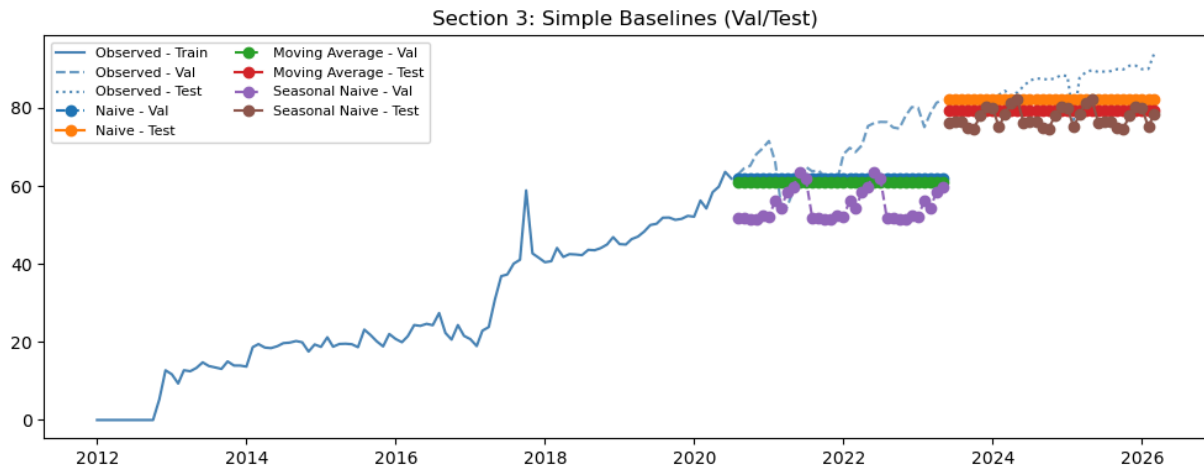
On this trimmed monthly series, these baselines are useful for establishing how much value comes from trend and seasonal structure alone. They remain interpretable, but the later surge in complaints means the flatter variants are expected to lag any model that adapts more aggressively to the post-2023 level shift.

```
In [4]: # =====
# SECTION 3: SIMPLE BASELINES
# =====
# Evaluate Naive, Moving Average, and Seasonal Naive forecasts

train_val = pd.concat([train, val]) # Combine train+val for test-set fitting
simple_pred_map = {} # Dictionary to store predictions for visualization

for name, val_pred, test_pred in [
    # Naive: repeat last value
    ("Naive", fc_naive(train, len(val)), fc_naive(train_val, len(test))),
    # Moving Average: repeat mean of last 4 values
    ("Moving Average", fc_mavg(train, len(val), k=4), fc_mavg(train_val, len(test),
    # Seasonal Naive: repeat last seasonal cycle (4 quarters)
    ("Seasonal Naive", fc_snaive(train, len(val), season=SEASONAL_PERIODS), fc_snai
]:
    t0 = time.perf_counter()
    runtime = time.perf_counter() - t0 # Simple methods are instantaneous
    append_result(name, val_pred, test_pred, runtime, fit_obj=None)
    simple_pred_map[name] = {"val": val_pred, "test": test_pred}

# Visualize predictions from all 3 baseline methods
plot_section_predictions("Section 3: Simple Baselines (Val/Test)", simple_pred_map)
print("Simple baselines evaluated.")
```



Simple baselines evaluated.

4. ETS (SES + Holt-Winters)

Exponential smoothing methods give more weight to recent observations, which is useful for this monthly share series because levels rise materially in the later years of the sample:

- **Simple Exponential Smoothing (SES):** tracks the current level, but cannot represent explicit trend or yearly seasonality
- **Holt-Winters Additive:** models level, trend, and additive 12-month seasonality simultaneously

In this notebook, ETS models remain important comparators, but they are no longer the leading benchmarks after full head-to-head evaluation against Theta and FFT on the held-out windows.

```
In [5]: # =====
# SECTION 4: EXPONENTIAL SMOOTHING (ETS)
# =====
# Evaluate Simple Exponential Smoothing (SES) and Holt-Winters additive smoothing

train_val = pd.concat([train, val]) # Combine train+val for test-set fitting
ets_pred_map = {} # Dictionary to store predictions for visualization

# ---- Simple Exponential Smoothing (SES) ----
t0 = time.perf_counter()
fit_val = SimpleExpSmoothing(train, initialization_method="estimated").fit()
val_pred = fit_val.forecast(len(val)).values
fit_test = SimpleExpSmoothing(train_val, initialization_method="estimated").fit()
test_pred = fit_test.forecast(len(test)).values
append_result("ETS SES", val_pred, test_pred, time.perf_counter() - t0, fit_obj=fit)
ets_pred_map["ETS SES"] = {"val": val_pred, "test": test_pred}

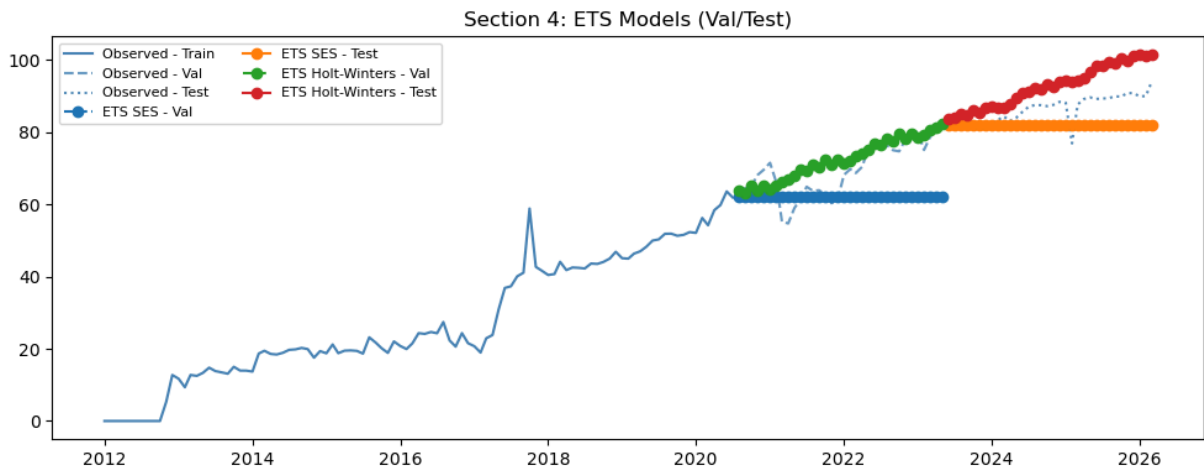
# ---- Holt-Winters Additive ----
t0 = time.perf_counter()
fit_val = ExponentialSmoothing(
    train, trend="add", seasonal="add", seasonal_periods=SEASONAL_PERIODS, initiali
).fit(optimized=True)
```

```

val_pred = fit_val.forecast(len(val)).values
fit_test = ExponentialSmoothing(
    train_val, trend="add", seasonal="add", seasonal_periods=SEASONAL_PERIODS, init
).fit(optimized=True)
test_pred = fit_test.forecast(len(test)).values
append_result("ETS Holt-Winters", val_pred, test_pred, time.perf_counter() - t0, fi
ets_pred_map["ETS Holt-Winters"] = {"val": val_pred, "test": test_pred}

# Visualize predictions from both ETS methods
plot_section_predictions("Section 4: ETS Models (Val/Test)", ets_pred_map)
print("ETS models evaluated.")

```



ETS models evaluated.

5. Evaluate ARIMA/SARIMA

ARIMA-family models combine autoregressive structure, differencing, and error correction to learn persistent dynamics in the monthly share series. Instead of fixing orders manually, a **stepwise AIC grid search** (implemented directly with statsmodels) selects the best parameter combination:

- **Auto ARIMA (non-seasonal)**: searches over (p, d, q) orders and picks the combination with the lowest AIC; serves as a compact non-seasonal benchmark that can absorb local trend changes
- **Auto SARIMA (seasonal)**: extends the search to seasonal $(P, D, Q, m=12)$ parameters so the recurring within-year structure in the monthly credit reporting share series is captured explicitly

For this dataset the seasonal variant is the relevant comparator because the trimmed series still shows recurring within-year structure on top of the broader rise in share values.

```

In [6]: # =====
# SECTION 5: ARIMA & SEASONAL ARIMA (SARIMA) – AIC grid search
# =====
# Lightweight AIC-based grid search using statsmodels directly (no external
# dependencies). Searches (p,d,q) and (P,D,Q,m) grids and selects the order
# with the lowest AIC, then evaluates on val/test as usual.

```

```

train_val = pd.concat([train, val])
arima_pred_map = {}
arima_orders = {}

def _aic_grid_arima(series, max_p=3, max_d=2, max_q=3):
    """Return the (p,d,q) order with the lowest AIC via exhaustive grid search."""
    best_aic, best_order = np.inf, (1, 1, 1)
    for p, d, q in product(range(max_p + 1), range(max_d + 1), range(max_q + 1)):
        if p == 0 and q == 0:
            continue
        try:
            aic = ARIMA(series, order=(p, d, q)).fit().aic
            if aic < best_aic:
                best_aic, best_order = aic, (p, d, q)
        except Exception:
            pass
    return best_order

def _aic_grid_sarima(series, order, m=12, max_P=2, max_D=1, max_Q=2):
    """Return the (P,D,Q,m) seasonal order with the lowest AIC given a fixed non-se
    best_aic, best_sorder = np.inf, (1, 1, 0, m)
    for P, D, Q in product(range(max_P + 1), range(max_D + 1), range(max_Q + 1)):
        if P == 0 and Q == 0:
            continue
        try:
            aic = SARIMAX(series, order=order, seasonal_order=(P, D, Q, m)).fit(dis
            if aic < best_aic:
                best_aic, best_sorder = aic, (P, D, Q, m)
        except Exception:
            pass
    return best_sorder

# ---- Non-seasonal ARIMA (AIC grid search) ----
print("Searching non-seasonal ARIMA orders...")
t0 = time.perf_counter()
arima_order = _aic_grid_arima(train)
arima_name = f"ARIMA{arima_order}"
print(f" \u2192 Selected: {arima_name}")
fit_val = ARIMA(train, order=arima_order).fit()
val_pred = fit_val.forecast(len(val)).values
fit_test = ARIMA(train_val, order=arima_order).fit()
test_pred = fit_test.forecast(len(test)).values
append_result(arima_name, val_pred, test_pred, time.perf_counter() - t0, fit_obj=fi
arima_pred_map[arima_name] = {"val": val_pred, "test": test_pred}
arima_orders[arima_name] = {"type": "arima", "order": arima_order}

# ---- Seasonal SARIMA (AIC grid search over seasonal part) ----
print("Searching seasonal SARIMA orders (this may take a moment)...")
t0 = time.perf_counter()
sarima_seasonal_order = _aic_grid_sarima(train, order=arima_order, m=SEASONAL_PERIO
P, D, Q, _ = sarima_seasonal_order
sarima_name = f"SARIMA{arima_order}x({P},{D},{Q},{SEASONAL_PERIODS})"
print(f" \u2192 Selected: {sarima_name}")
fit_val = SARIMAX(train, order=arima_order, seasonal_order=sarima_seasonal_ord
val_pred = fit_val.forecast(len(val)).values

```

```

fit_test = SARIMAX(train_val, order=arima_order, seasonal_order=sarima_seasonal_order)
test_pred = fit_test.forecast(len(test)).values
append_result(sarima_name, val_pred, test_pred, time.perf_counter() - t0, fit_obj=fit_test)
arima_pred_map[sarima_name] = {"val": val_pred, "test": test_pred}
arima_orders[sarima_name] = {"type": "sarima", "order": arima_order, "seasonal_order": sarima_seasonal_order}

# Visualize predictions from both ARIMA methods
plot_section_predictions("Section 5: ARIMA/SARIMA (Val/Test)", arima_pred_map)
print("ARIMA/SARIMA evaluated.")

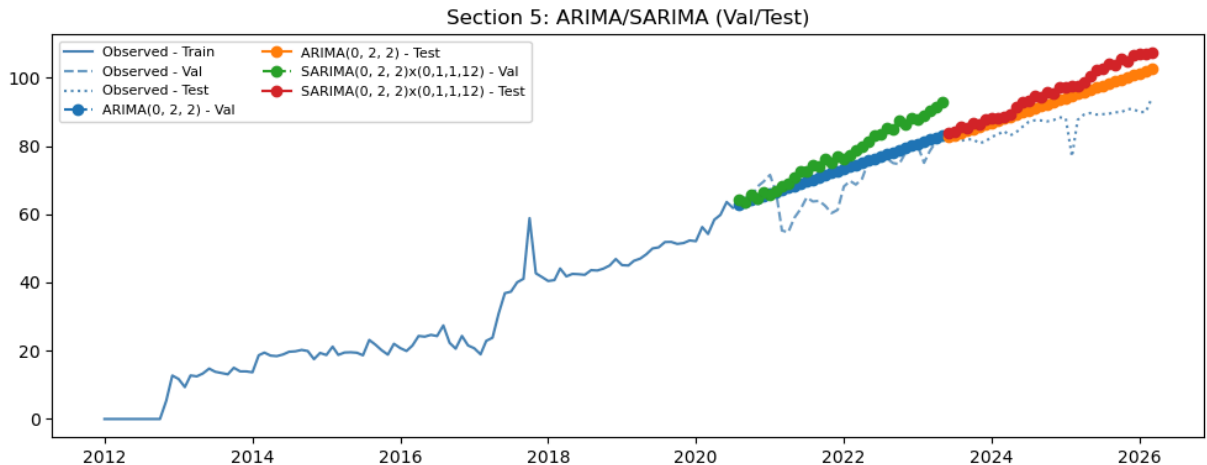
```

Searching non-seasonal ARIMA orders...

→ Selected: ARIMA(0, 2, 2)

Searching seasonal SARIMA orders (this may take a moment)...

→ Selected: SARIMA(0, 2, 2)x(0,1,1,12)



ARIMA/SARIMA evaluated.

6. Theta

The Theta method is a practical monthly forecasting baseline because it captures smooth trend behavior after deseasonalization without requiring a full seasonal state-space specification.

It works by deseasonalizing the series, fitting a Theta-style extrapolation to the adjusted signal, and then reapplying the seasonal pattern. In the current run, Theta ranks among the strongest models and serves as the primary interpretable benchmark for this credit reporting share series.

```

In [7]: # =====
# SECTION 6: THETA METHOD
# =====
# Evaluate Theta model with deseasonalization (effective for seasonal data)

train_val = pd.concat([train, val]) # Combine train+val for test-set fitting
theta_pred_map = {} # Dictionary to store predictions for visualization

# Theta decomposes series into level + season before forecasting
# Works well for quarterly/seasonal data
t0 = time.perf_counter()
fit_val = ThetaModel(train, period=SEASONAL_PERIODS, deseasonalize=True).fit()

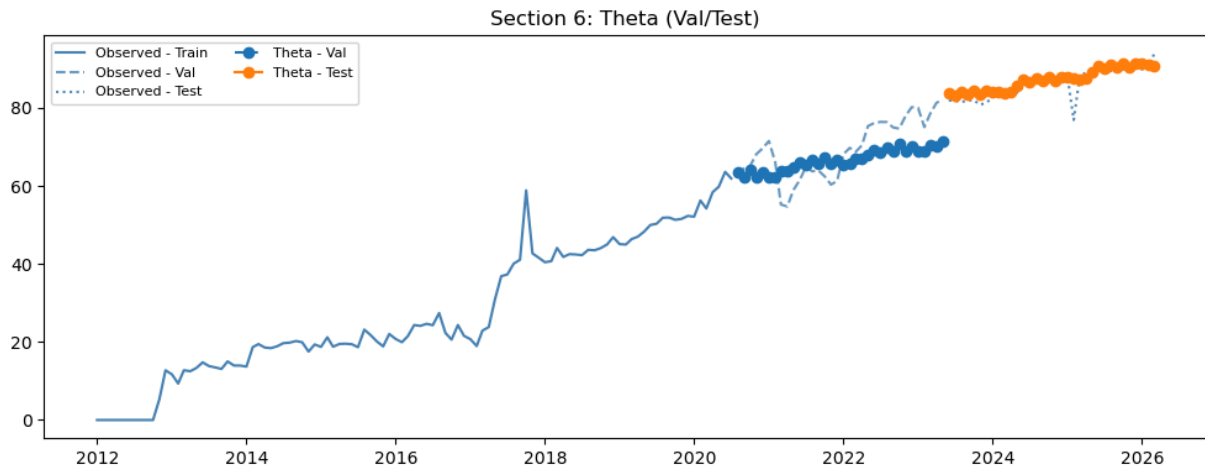
```

```

val_pred = fit_val.forecast(len(val)).values
fit_test = ThetaModel(train_val, period=SEASONAL_PERIODS, deseasonalize=True).fit()
test_pred = fit_test.forecast(len(test)).values
append_result("Theta", val_pred, test_pred, time.perf_counter() - t0, fit_obj=fit_t)
theta_pred_map["Theta"] = {"val": val_pred, "test": test_pred}

# Visualize Theta predictions vs observed
plot_section_predictions("Section 6: Theta (Val/Test)", theta_pred_map)
print("Theta evaluated.")

```



Theta evaluated.

7. FFT

The Fast Fourier Transform (FFT) takes a frequency-domain approach to forecasting:

1. **Detrend** the series (remove linear trend)
2. **Compute FFT** to extract the dominant frequency components
3. **Retain top 6 frequencies** (largest amplitudes) to focus on the most significant seasonal and cyclical patterns
4. **Reconstruct** the signal in the future horizon using only these key frequencies
5. **Re-trend** by adding back the linear trend component

This method discovers dominant cycles empirically and is useful when the seasonal pattern is strong and stable. It complements statistical methods by exploiting periodicity without distributional assumptions.

```

In [8]: # =====
# SECTION 7: FAST FOURIER TRANSFORM (FFT)
# =====
# Evaluate FFT-based frequency reconstruction for seasonal forecasting

train_val = pd.concat([train, val]) # Combine train+val for test-set fitting
fft_pred_map = {} # Dictionary to store predictions for visualization

# FFT method: extract top 6 frequency components and reconstruct for forecast horizon
t0 = time.perf_counter()
val_pred = fc_fft(train, len(val), keep=6) # Forecast validation set

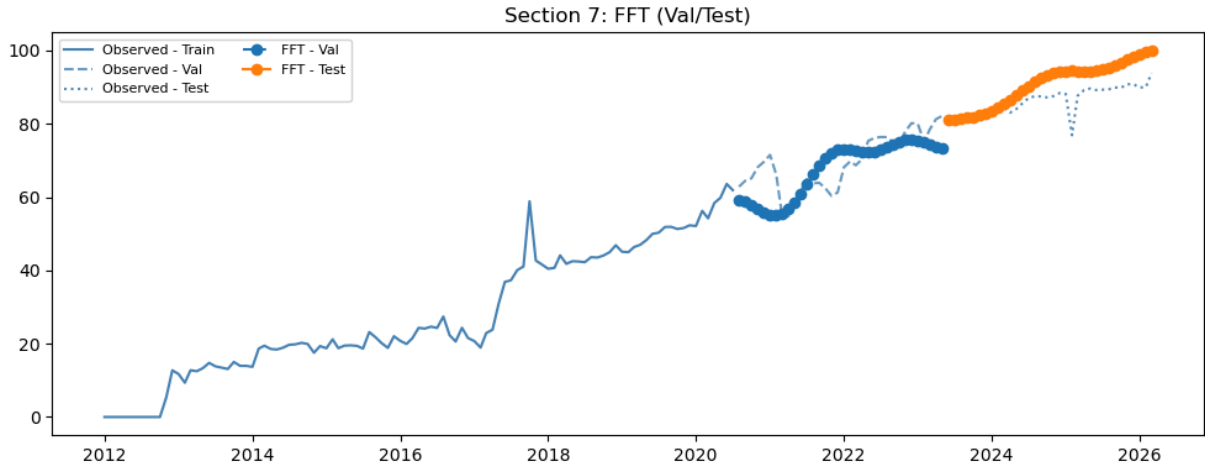
```

```

test_pred = fc_fft(train_val, len(test), keep=6) # Forecast test set
append_result("FFT", val_pred, test_pred, time.perf_counter() - t0, fit_obj=None)
fft_pred_map["FFT"] = {"val": val_pred, "test": test_pred}

# Visualize FFT predictions vs observed
plot_section_predictions("Section 7: FFT (Val/Test)", fft_pred_map)
print("FFT evaluated.")

```



FFT evaluated.

8. Comparison Table

This table ranks all monthly baselines on the held-out test window using sMAPE, MAE, RMSE, and runtime for the trimmed 2012-2025 credit reporting share series.

The ranking is informative beyond the top two models: flatter methods can miss regime shifts, while stronger seasonality-aware models stay materially closer to the observed monthly path.

```

In [9]: # =====
# SECTION 8: COMPARISON TABLE
# =====
# Aggregate all model results, add interpretive notes, and rank by test performance

# Define interpretive notes for each model based on its expected behavior
notes_map = {
    "Naive": "Too biased",
    "Moving Average": "Too flat",
    "Seasonal Naive": "Misses trend/regime shifts",
    "ETS SES": "May underfit trend",
    "ETS Holt-Winters": "Can over/under-smooth peaks",
    "arima_name": "No explicit seasonality (auto-selected orders)",
    "sarima_name": "Strong seasonal baseline (auto-selected orders)",
    "Theta": "Strong trend-seasonality balance",
    "FFT": "Seasonal frequency focus"
}

# Convert eval_rows list to DataFrame and attach notes
results = pd.DataFrame(eval_rows)

```

```
results["Notes"] = results["Model"].map(notes_map).fillna("")

# Sort by test performance (primary: sMAPE_test, secondary: MAE_test, tertiary: run
results = results.sort_values(["sMAPE_test", "MAE_test", "runtime_sec"]).reset_inde

# Select columns for display and round metrics to 3 decimal places
comparison_cols = ["Model", "sMAPE_val", "MAE_val", "RMSE_val", "sMAPE_test", "MAE_
results_display = results[comparison_cols].copy()
for c in ["sMAPE_val", "MAE_val", "RMSE_val", "sMAPE_test", "MAE_test", "RMSE_test"
    results_display[c] = results_display[c].round(3)

# Display the comparison table
display(results_display)
```

	Model	sMAPE_val	MAE_val	RMSE_val	sMAPE_test	MAE_test	RMSE_test	runtime_s
0	Theta	8.266	5.698	6.466	1.788	1.525	2.361	0.0
1	FFT	7.573	5.081	6.584	5.132	4.652	5.791	0.0
2	Naive	12.776	8.674	10.652	5.490	4.684	5.648	0.0
3	ETS SES	12.603	8.567	10.535	5.566	4.746	5.735	0.0
4	ARIMA(0, 2, 2)	6.582	4.457	5.859	6.957	6.321	7.190	10.4
5	ETS Holt-Winters	6.217	4.165	5.577	7.140	6.482	7.258	0.2
6	Moving Average	13.886	9.362	11.365	8.603	7.197	8.030	0.0
7	SARIMA(0, 2, 2)x(0,1,1,12)	11.891	8.713	9.635	10.064	9.351	10.477	54.3
8	Seasonal Naive	23.439	14.824	16.640	10.578	8.716	9.677	0.0

9. Assess Forecastability

The trimmed monthly credit reporting share series is best described as **moderately forecastable**, not easy. The current diagnostics show:

- **CoV** reflects meaningful variability in monthly share levels
- **Residual Variability** indicates there is remaining noise after trend-seasonal adjustment
- **Spectral Entropy** remains far from white noise, supporting structured seasonal baselines

Taken together, these diagnostics explain why stronger monthly models can forecast credibly while still penalizing methods that underreact to the recent regime shift.

```
In [10]: # =====
# SECTION 9: FORECASTABILITY ASSESSMENT
# =====
# Compute objective metrics to assess how predictable the series is

# ---- Coefficient of Variation (CoV) ----
# Ratio of standard deviation to mean; <0.5 = easy, >0.8 = hard to forecast
cov = float(y.std(ddof=1) / y.mean())
if cov < 0.5:
    cov_label = "easy"
elif cov > 0.8:
    cov_label = "hard"
else:
    cov_label = "moderate"

# ---- Residual Variability (RV) ----
# Variability remaining after removing trend and seasonality via decomposition
if len(y) >= 2 * 4:
    decomp = seasonal_decompose(y.to_timestamp(how="end"), model="additive", period
    resid = pd.Series(decomp.resid).dropna()
    rv = float(resid.std(ddof=1) / y.mean()) if len(resid) else np.nan
else:
    rv = np.nan

# ---- Spectral Entropy ----
# Normalized entropy of power spectral density (0-1 scale)
# Higher entropy = more uniform frequency distribution = less predictable dynamics
spectral_entropy = compute_spectral_entropy(y.values)

# Compile forecastability metrics into interpretable DataFrame
forecastability_df = pd.DataFrame([
    {"Metric": "CoV", "Value": cov, "Interpretation": f"{cov_label} ({'<0.5 easy, >
    {"Metric": "Residual Variability", "Value": rv, "Interpretation": "Lower is bet
    {"Metric": "Spectral Entropy", "Value": spectral_entropy, "Interpretation": "Hi
])

# Display the forecastability assessment
display(forecastability_df.round(4))
```

	Metric	Value	Interpretation
0	CoV	0.6041	moderate (<0.5 easy, >0.8 hard)
1	Residual Variability	0.0579	Lower is better after trend/seasonality removal
2	Spectral Entropy	0.3411	Higher implies less predictable dynamics

10. Select and Plot Top Baselines

The current monthly run selects **Theta** and **FFT** as the two strongest baselines on the held-out test window.

- **Theta** is the primary benchmark due to its strong test accuracy and stable regional behavior
- **FFT** is a fast, frequency-based challenger that captures cyclical structure well but can extrapolate aggressively on bounded share targets
- The 12-month forward forecasts stay elevated for both regions, with Theta generally producing more conservative and plausible share paths

Residual diagnostics are still used to confirm that top-ranked models are not leaving obvious structure behind before treating them as benchmark forecasts.

```
In [11]: # =====
# SECTION 10: BEST BASELINE PLOT & SELECTION
# =====
# Extract top 2 models (by test performance), run diagnostics, and produce forward

# ---- Residual Diagnostics for Top 2 Models ----
# Check residual mean/std and Ljung-Box autocorrelation test (Lag 4)
diag_df = run_residual_diagnostics(results, model_registry, top_n=2)
print("Residual Diagnostics for Top 2 Models:")
display(diag_df.round(4))

# ---- Select Top 2 Models and Generate 4-Quarter Forward Forecast ----
selected = results.head(2)["Model"].tolist() # Get best 2 by SMAPE_test, MAE_test,
h = FORECAST_HORIZON
future_index = pd.period_range(start=y.index[-1] + 1, periods=h, freq=FREQ_ALIAS)

# Re-fit each selected model on full data and generate future forecast
forecast_tbl = pd.DataFrame(index=future_index.astype(str))
for m in selected:
    # Re-fit model on full series (train+val+test) for production forecast
    if m == "Naive":
        fc = fc_naive(y, h)
    elif m == "Moving Average":
        fc = fc_mavg(y, h, k=4)
    elif m == "Seasonal Naive":
        fc = fc_snaive(y, h, season=SEASONAL_PERIODS)
    elif m == "ETS SES":
        fc = SimpleExpSmoothing(y, initialization_method="estimated").fit().forecas
```

```

elif m == "ETS Holt-Winters":
    fc = ExponentialSmoothing(y, trend="add", seasonal="add", seasonal_periods=
elif m in arima_orders:
    info = arima_orders[m]
    if info["type"] == "arima":
        fc = ARIMA(y, order=info["order"]).fit().forecast(h).values
    else:
        fc = SARIMAX(y, order=info["order"], seasonal_order=info["seasonal_order"]
elif m == "Theta":
    fc = ThetaModel(y, period=SEASONAL_PERIODS, deseasonalize=True).fit().forec
elif m == "FFT":
    fc = fc_fft(y, h, keep=6)
else:
    continue
forecast_tbl[m] = np.round(fc, 1)

print("\n12-Month Forward Forecast from Top 2 Baselines:")
display(forecast_tbl)

# ---- Plot Observed Series + Top 2 Baseline Forecasts ----
fig, ax = plt.subplots(figsize=(10, 4))
y_ts = pd.Series(y.values, index=y.index.to_timestamp(how="end"))
ax.plot(y_ts.index, y_ts.values, label="Observed", color="steelblue")

for m in selected:
    idx = future_index.to_timestamp(how="end")
    ax.plot(idx, forecast_tbl[m].values, marker="o", label=f"Forecast - {m}")

ax.set_title("Top Baseline Forecasts (Next 12 Months)")
ax.legend()
plt.tight_layout()
plt.show()

# Print summary
print("\nSelected strong baselines:", selected)
print("Rationale: lowest test sMAPE/MAE with reasonable runtime, then checked diagn

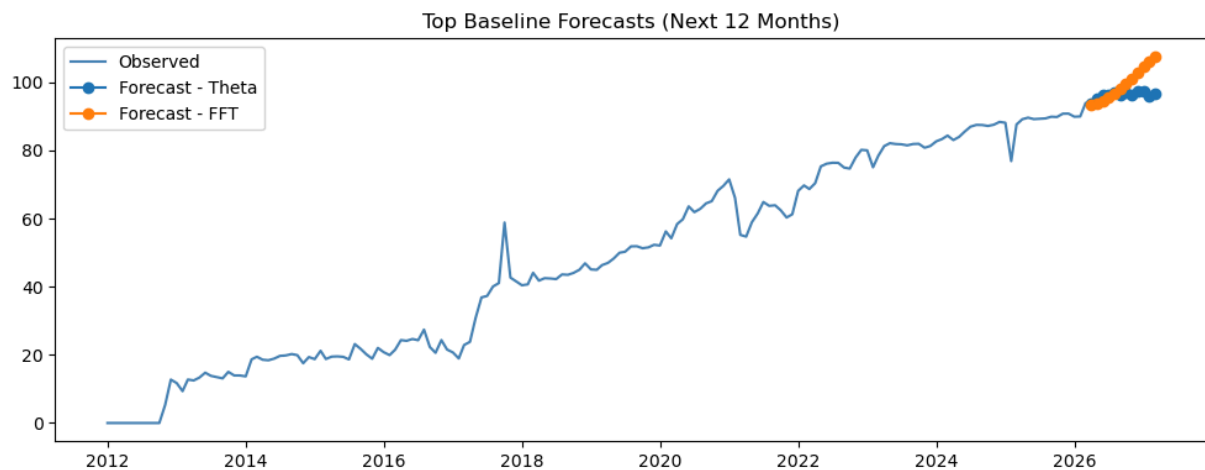
```

Residual Diagnostics for Top 2 Models:

	Model	resid_mean	resid_std	ljung_box_p_lag4
0	Theta	NaN	NaN	NaN
1	FFT	NaN	NaN	NaN

12-Month Forward Forecast from Top 2 Baselines:

	Theta	FFT
2026-03	93.8	93.5
2026-04	95.3	93.9
2026-05	96.4	94.6
2026-06	96.2	95.5
2026-07	96.9	96.6
2026-08	96.4	98.0
2026-09	97.2	99.5
2026-10	96.4	101.2
2026-11	97.4	102.9
2026-12	97.3	104.5
2027-01	96.1	106.1
2027-02	96.8	107.5



Selected strong baselines: ['Theta', 'FFT']

Rationale: lowest test sMAPE/MAE with reasonable runtime, then checked diagnostics for top models.

10A. Auto Interpretation

```
In [12]: # =====
# SECTION 11: AUTO INTERPRETATION (MODEL-SPECIFIC)
# =====
# Generate automated summary of model selection: which models won and why others Lo

# Identify selected (top 2) and non-selected models
selected_models = results.head(2)["Model"].tolist()
selected_set = set(selected_models)

# Get best model's performance for comparison
best_smape = float(results.iloc[0]["sMAPE_test"])
```

```

best_mae = float(results.iloc[0]["MAE_test"])

# ---- Print Selected Baselines ----
print("Strong baselines selected:")
for i, m in enumerate(selected_models, start=1):
    row = results[results["Model"] == m].iloc[0]
    print(f"{i}. {m} | sMAPE_test={row['sMAPE_test']:.2f}, MAE_test={row['MAE_test']:.2f}, runtime={row['runtime']:.3f}s")

# ---- Explain Why Other Models Lost ----
print("\nWhy other models lost:")
losers = results[~results["Model"].isin(selected_set)].copy()
for _, row in losers.iterrows():
    gap_smape = float(row["sMAPE_test"] - best_smape)
    gap_mae = float(row["MAE_test"] - best_mae)
    note = row.get("Notes", "")
    print(
        f"- {row['Model']}: {note}; +{gap_smape:.2f} sMAPE and +{gap_mae:.1f} MAE vs best model."
    )

# ---- Explain Selection Rationale ----
print("\nSelection rationale:")
print("Lowest test sMAPE/MAE first, then runtime and residual diagnostics for the top candidates.")

```

Strong baselines selected:

1. Theta | sMAPE_test=1.79, MAE_test=1.5, runtime=0.0795s
2. FFT | sMAPE_test=5.13, MAE_test=4.7, runtime=0.0032s

Why other models lost:

- Naive: Too biased; +3.70 sMAPE and +3.2 MAE vs best model.
- ETS SES: May underfit trend; +3.78 sMAPE and +3.2 MAE vs best model.
- ARIMA(0, 2, 2): No explicit seasonality (auto-selected orders); +5.17 sMAPE and +4.8 MAE vs best model.
- ETS Holt-Winters: Can over/under-smooth peaks; +5.35 sMAPE and +5.0 MAE vs best model.
- Moving Average: Too flat; +6.82 sMAPE and +5.7 MAE vs best model.
- SARIMA(0, 2, 2)x(0,1,1,12): Strong seasonal baseline (auto-selected orders); +8.28 sMAPE and +7.8 MAE vs best model.
- Seasonal Naive: Misses trend/regime shifts; +8.79 sMAPE and +7.2 MAE vs best model.

Selection rationale:

Lowest test sMAPE/MAE first, then runtime and residual diagnostics for the top candidates.

11. Conclusion

Using the trimmed monthly credit reporting share series from 2012-01-01 through 2025-12-31, this notebook establishes a focused classical benchmark for CFPB credit reporting share forecasting.

Key Findings:

- **Best Monthly Baseline:** Theta (top held-out performance)
- **Runner-Up:** FFT as the main alternative benchmark

- **Forecastability:** assessed with CoV, Residual Variability, and Spectral Entropy diagnostics
- **Behavioral Pattern:** simpler and flatter baselines remain interpretable reference points, but often underreact to late-sample acceleration

Implication: Theta is the benchmark to carry forward for regional and downstream modeling, with FFT retained as a comparison model. Because the target is a percentage share, forecast plausibility checks (for example boundedness) should be applied when interpreting longer-horizon FFT projections.

12. Compare South vs Non-South (Share % TSF)

This section runs regional time-series forecasting on **share data** (`credit_reporting_share_pct`) and separates model evaluation into two parts:

- **12A (Best model):** Theta
- **12B (Second-best model):** FFT

Each model is run independently for South and Non-South series with train/validation/test evaluation and 12-month forward forecasts.

```
In [14]: # =====
# SECTION 12: SOUTH VS NON-SOUTH TSF (SHARE DATA ONLY)
# =====
# 12A: Best model -> Theta
# 12B: 2nd-best model -> FFT

TARGET_COL = "credit_reporting_share_pct"

south_path = "../data/processed/credit_reporting_monthly_south_data.csv"
non_south_path = "../data/processed/credit_reporting_monthly_non_south_data.csv"

def load_monthly_series(path, value_col):
    df_reg = pd.read_csv(path)
    df_reg["month"] = pd.PeriodIndex(df_reg["month"], freq="M")
    df_reg = df_reg.sort_values("month")
    return pd.Series(df_reg[value_col].astype(float).values, index=df_reg["month"],

def split_series(y_series):
    n = len(y_series)
    test_size = max(4, n // 5)
    val_size = test_size
    while n - (val_size + test_size) < 8 and test_size > 2:
        test_size -= 1
        val_size = test_size
    train_s = y_series.iloc[: n - val_size - test_size].copy()
    val_s = y_series.iloc[n - val_size - test_size : n - test_size].copy()
    test_s = y_series.iloc[n - test_size :].copy()
```

```

    return train_s, val_s, test_s

def fit_predict_model(train_s, train_val_s, test_len, val_len, model_name):
    if model_name == "Theta":
        fit_val = ThetaModel(train_s, period=SEASONAL_PERIODS, deseasonalize=True).fit(train_s, train_val_s)
        val_pred = fit_val.forecast(val_len).values
        fit_test = ThetaModel(train_val_s, period=SEASONAL_PERIODS, deseasonalize=True).fit(train_val_s, test_s)
        test_pred = fit_test.forecast(test_len).values
        return val_pred, test_pred

    if model_name == "FFT":
        val_pred = fc_fft(train_s, val_len, keep=6)
        test_pred = fc_fft(train_val_s, test_len, keep=6)
        return val_pred, test_pred

    raise ValueError(f"Unsupported model: {model_name}")

def forward_forecast(y_series, horizon, model_name):
    if model_name == "Theta":
        fc = ThetaModel(y_series, period=SEASONAL_PERIODS, deseasonalize=True).fit(y_series)
        return np.round(fc, 4)

    if model_name == "FFT":
        fc = fc_fft(y_series, horizon, keep=6)
        return np.round(fc, 4)

    raise ValueError(f"Unsupported model: {model_name}")

def plot_region_model(region_name, model_name, train_s, val_s, test_s, val_pred, test_pred):
    fig, ax = plt.subplots(figsize=(10, 4))
    train_ts = pd.Series(train_s.values, index=train_s.index.to_timestamp(how="end"))
    val_ts = pd.Series(val_s.values, index=val_s.index.to_timestamp(how="end"))
    test_ts = pd.Series(test_s.values, index=test_s.index.to_timestamp(how="end"))

    ax.plot(train_ts.index, train_ts.values, color="steelblue", label="Observed - Train")
    ax.plot(val_ts.index, val_ts.values, color="steelblue", linestyle="--", alpha=0.5, label="Observed - Validation")
    ax.plot(test_ts.index, test_ts.values, color="steelblue", linestyle=":", alpha=0.5, label="Observed - Test")

    val_pred_ts = pd.Series(val_pred, index=val_s.index.to_timestamp(how="end"))
    test_pred_ts = pd.Series(test_pred, index=test_s.index.to_timestamp(how="end"))
    ax.plot(val_pred_ts.index, val_pred_ts.values, linestyle="--", marker="o", label=f"Predicted - Validation ({model_name})")
    ax.plot(test_pred_ts.index, test_pred_ts.values, marker="o", label=f"Predicted - Test ({model_name})")

    ax.set_title(f"{region_name}: {model_name} (Share % TSF)")
    ax.legend(ncol=2, fontsize=8)
    plt.tight_layout()
    plt.show()

def evaluate_model_by_region(model_name, region_series_map, horizon=FORECAST_HORIZON):
    rows = []
    forecast_tbl = None

    for region_name, y_series in region_series_map.items():
        train_s, val_s, test_s = split_series(y_series)
        train_val_s = pd.concat([train_s, val_s])
        val_pred, test_pred = fit_predict_model(train_s, train_val_s, test_s, horizon, model_name)
        rows.append([region_name, model_name, val_pred, test_pred])

    forecast_tbl = pd.DataFrame(rows)
    return forecast_tbl

```

```

t0 = time.perf_counter()
val_pred, test_pred = fit_predict_model(
    train_s, train_val_s, len(test_s), len(val_s), model_name
)
runtime = time.perf_counter() - t0

rows.append({
    "Region": region_name,
    "Model": model_name,
    "sMAPE_val": smape(val_s.values, val_pred),
    "MAE_val": mae(val_s.values, val_pred),
    "RMSE_val": rmse(val_s.values, val_pred),
    "sMAPE_test": smape(test_s.values, test_pred),
    "MAE_test": mae(test_s.values, test_pred),
    "RMSE_test": rmse(test_s.values, test_pred),
    "runtime_sec": runtime,
})

plot_region_model(region_name, model_name, train_s, val_s, test_s, val_pred)

future_index = pd.period_range(start=y_series.index[-1] + 1, periods=horizon)
if forecast_tbl is None:
    forecast_tbl = pd.DataFrame(index=future_index.astype(str))
forecast_tbl[region_name] = forward_forecast(y_series, horizon, model_name)

metrics_df = pd.DataFrame(rows)
for c in ["sMAPE_val", "MAE_val", "RMSE_val", "sMAPE_test", "MAE_test", "RMSE_test"]:
    metrics_df[c] = metrics_df[c].round(4)

return metrics_df, forecast_tbl

# Load South / Non-South share series
y_south = load_monthly_series(south_path, TARGET_COL)
y_non_south = load_monthly_series(non_south_path, TARGET_COL)
region_series = {"South": y_south, "Non-South": y_non_south}

# -----
# 12A. Best model: Theta
# -----
print("\n12A. Best model: Theta (share % TSF)")
theta_regional_metrics, theta_regional_fc = evaluate_model_by_region("Theta", region_series)
display(theta_regional_metrics.sort_values(["Region"]).reset_index(drop=True))

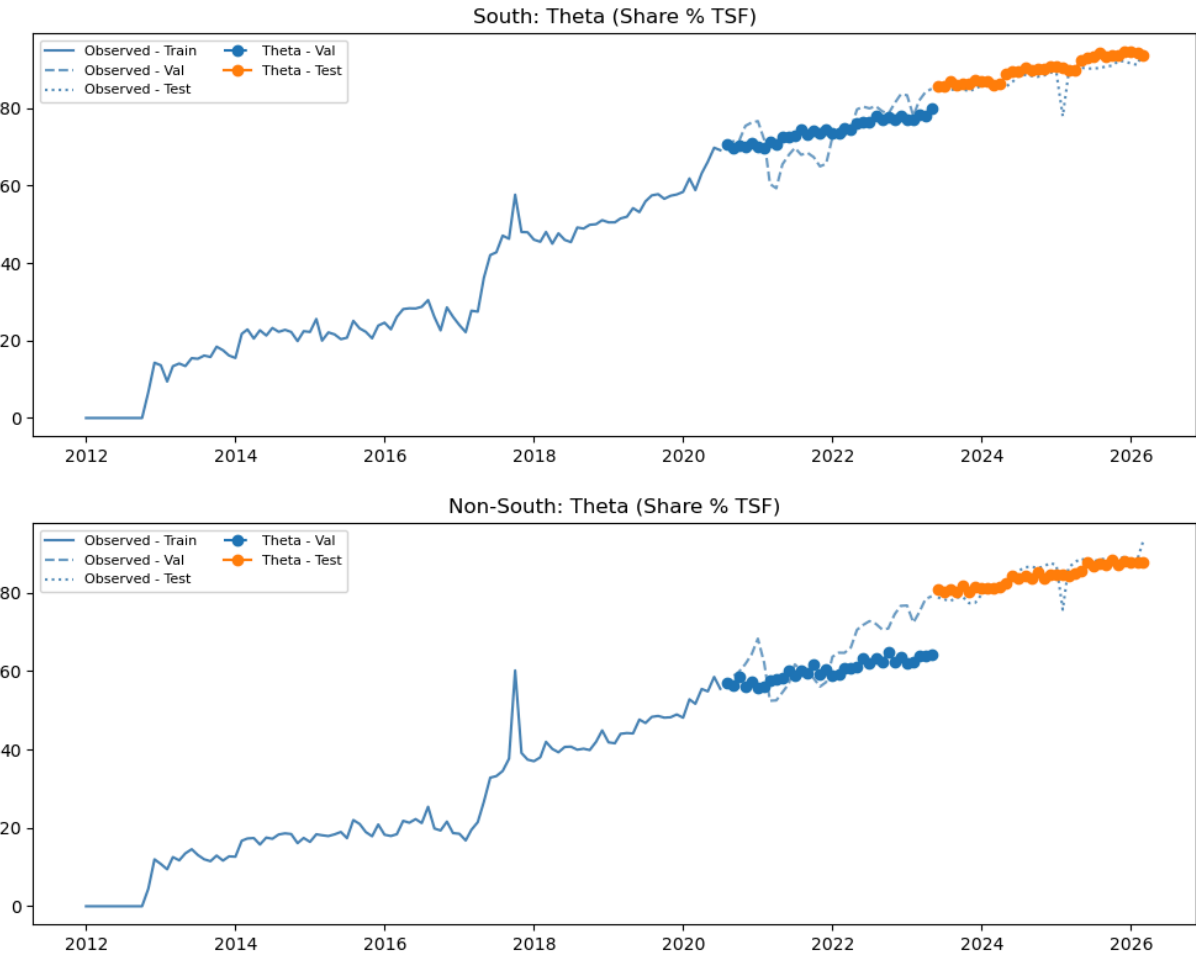
print("\n12-month forward forecast (Theta) - share %")
display(theta_regional_fc)

# -----
# 12B. Second-best model: FFT
# -----
print("\n12B. 2nd-best model: FFT (share % TSF)")
fft_regional_metrics, fft_regional_fc = evaluate_model_by_region("FFT", region_series)
display(fft_regional_metrics.sort_values(["Region"]).reset_index(drop=True))

print("\n12-month forward forecast (FFT) - share %")
display(fft_regional_fc)

```

12A. Best model: Theta (share % TSF)

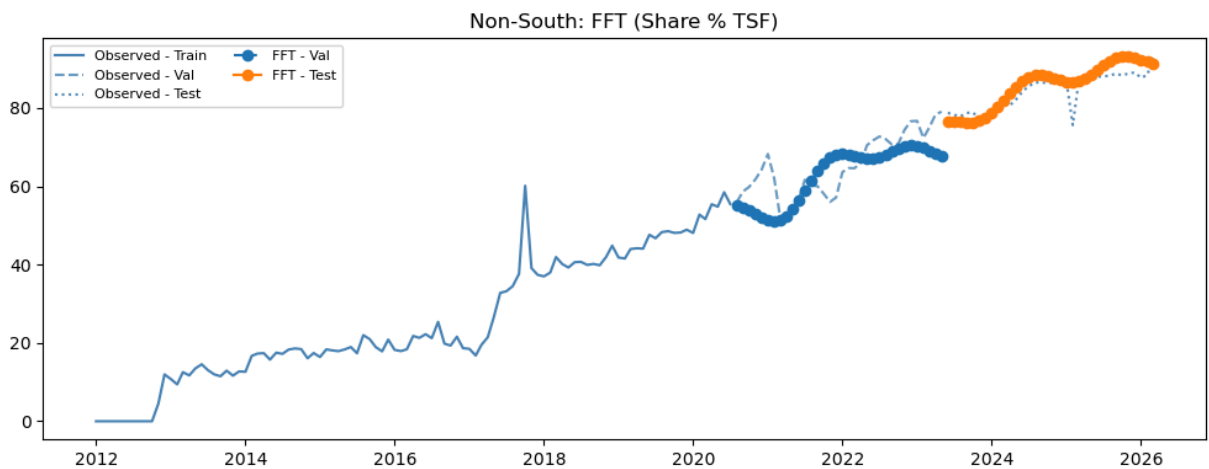
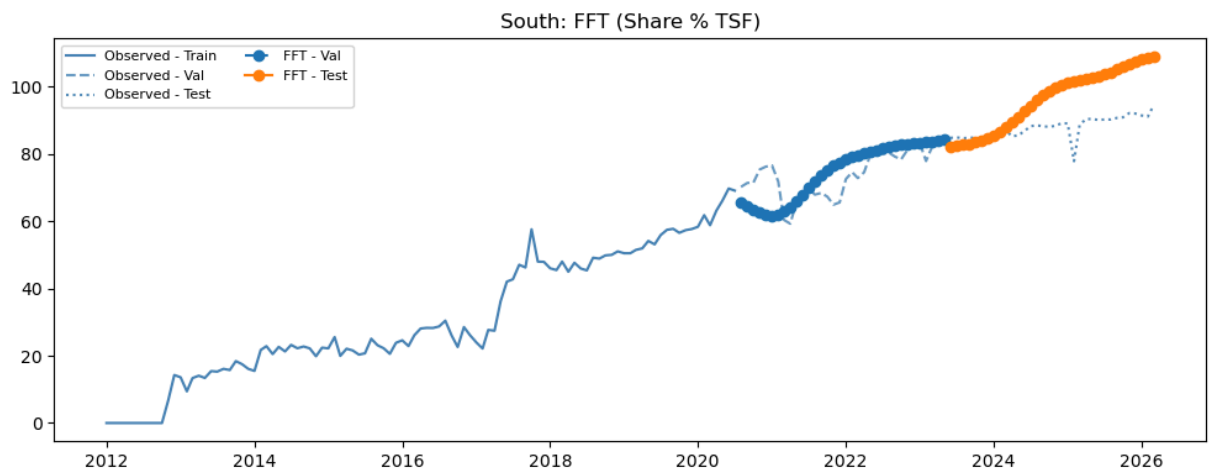


	Region	Model	sMAPE_val	MAE_val	RMSE_val	sMAPE_test	MAE_test	RMSE_test	runti
0	Non-South	Theta	10.3318	6.7125	7.9403	2.4349	2.0331	2.6471	
1	South	Theta	5.9707	4.3392	5.2105	2.4327	2.1577	2.9532	



	South	Non-South
2026-03	94.3923	92.7354
2026-04	96.4397	93.5005
2026-05	96.9779	95.0822
2026-06	97.1669	94.6032
2026-07	98.0101	95.0669
2026-08	97.3066	94.7772
2026-09	97.5898	95.9999
2026-10	97.5365	94.5999
2026-11	98.4918	95.7497
2026-12	98.4432	95.6820
2027-01	97.0613	94.4831
2027-02	97.6365	95.4787

12B. 2nd-best model: FFT (share % TSF)



	Region	Model	sMAPE_val	MAE_val	RMSE_val	sMAPE_test	MAE_test	RMSE_test	runtii
0	Non-South	FFT	8.2224	5.2548	6.6678	2.6557	2.274	3.0667	
1	South	FFT	6.8322	4.8711	6.4880	9.4719	9.034	10.9672	



12-month forward forecast (FFT) - share %

	South	Non-South
2026-03	94.7028	91.8424
2026-04	95.0946	92.2207
2026-05	95.7337	92.7958
2026-06	96.6426	93.5939
2026-07	97.8185	94.6216
2026-08	99.2339	95.8651
2026-09	100.8388	97.2903
2026-10	102.5658	98.8457
2026-11	104.3363	100.4668
2026-12	106.0685	102.0821
2027-01	107.6854	103.6195
2027-02	109.1218	105.0124